

Final Project Report

CS 6800: Distributed Systems Theory and Analysis

Scalable Distributed Training for Hierarchical Cross-View Geo-Localization

Architecture, Benchmarking, and Fault Tolerance Analysis of
Multi-GPU DDP Training for HCVGLoc

Ganesh Sapkota

Department of Computer Science Missouri University of Science and Technology

Abstract

Cross-view geo-localization for GPS-denied UAV navigation demands computationally intensive training pipelines that benefit significantly from multi-GPU parallelism. This report presents the design, implementation, and empirical evaluation of a scalable distributed training infrastructure for HCVGLoc, a hierarchical drone-to-satellite image retrieval framework. We extend the HCVGLoc Stage 1 coarse-retrieval pipeline to a multi-GPU setting using PyTorch Distributed Data Parallel (DDP) with NCCL all-reduce synchronization across four NVIDIA RTX 6000 Ada GPUs, and conduct a systematic study across four evaluation pillars. **Scalability:** A 2-D weak-scaling sweep ($W \in \{1, 2, 4\}$, $B \in \{16, 32, 64, 128\}$) yields parallel efficiency of 72.8 – 94.7% at $W = 4$, demonstrating a clean communication-bound to compute-bound transition with increasing batch size. **Communication overhead:** AllReduce cost is effectively free at $W = 2$ ($\leq 12\%$) and reaches a best-case of 18.8% at $W = 4$, $B = 64$. **Fault tolerance:** Checkpoint I/O overhead analysis confirms the 10-epoch default interval is near-optimal on the overhead-vs-steps-lost Pareto curve (0.36% overhead, ~ 5 average steps lost per failure). **Convergence parity:** Three 20-epoch runs at $W \in \{1, 2, 4\}$ produce statistically indistinguishable R@K metrics, confirming that the convergence bottleneck is the loss formulation rather than DDP parallelism. Training-only speedup between $W = 2$ and $W = 4$ is 95.2% efficient; cumulative speedup (69%) is reduced by un-sharded validation, a tractable engineering fix identified for future work.

The code is available at github.com/gxanesh/distributed-HCVGLoc.

1. Introduction

Large-scale cross-view geo-localization systems, which match unmanned aerial vehicle (UAV) imagery against geo-referenced satellite databases for GPS-denied localization, impose substantial computational demands during training. The HCVGLoc framework – a hierarchical pipeline comprising contrastive metric learning for coarse retrieval, fine-grained pose estimation, and Extended Kalman Filter (EKF) temporal fusion – requires processing millions of cross-view image pairs across multi-scale feature extraction backbones with

adversarial domain alignment. Training such a system on a single GPU is prohibitively slow, motivating the adoption of distributed training strategies.

This project formalizes and rigorously evaluates the distributed training design of HCVGLoc, which employs PyTorch Distributed Data Parallel (DDP) with NCCL-based all-reduce gradient synchronization across four NVIDIA RTX Ada 6000 GPUs. Beyond simply deploying multi-GPU training, this work treats the distributed training infrastructure as a first-class distributed system and performs a systematic study across four evaluation pillars:

- (i) Scalability benchmarking with linear speedup analysis across a 2-D grid of GPU counts and batch sizes,
- (ii) Communication overhead profiling of AllReduce synchronization versus compute time,
- (iii) Fault tolerance through checkpoint-based recovery under simulated GPU failures, and
- (iv) Controlled comparison of distributed versus single-GPU training on convergence, throughput, and retrieval accuracy (R@1, R@5, R@10).

The contributions of this work include: (a) a full torchrun-compatible DDP training stack for a multi-head geo-localization model with adversarial domain adaptation; (b) a 2-D weak-scaling benchmark correcting methodological errors in single-baseline efficiency measurement; (c) quantitative evidence that training-only DDP speedup is near-ideal (95.2% at $W = 4$, $B = 64$) while cumulative speedup is bounded by un-sharded validation; and (d) a convergence parity study showing that R@K is statistically indistinguishable across world sizes 1, 2, and 4, confirming that the loss formulation rather than parallelism is the convergence bottleneck.

2. Related Work

Distributed deep learning training has evolved from the Parameter Server architecture [1] – where a central server aggregates and redistributes gradients from worker nodes – to fully decentralized approaches such as ring all-reduce [2], which eliminates the server bottleneck by passing gradients in a ring topology. The Horovod library [3] demonstrated that ring all-reduce enables near-linear scaling across hundreds of GPUs with minimal code changes. PyTorch’s native DistributedDataParallel (DDP) [4] builds on these principles by overlapping gradient computation with all-reduce communication through bucketed synchronization over the NCCL backend, and has become the standard for data-parallel training on multi-GPU workstations and clusters.

Fault tolerance in distributed ML is typically addressed through two complementary mechanisms. Checkpoint-restart [6] – the simplest and most widely deployed approach – saves full training state at regular intervals, trading periodic I/O overhead for bounded re-computation on failure. Elastic training frameworks such as TorchElastic [5] go further, allowing worker processes to dynamically join or leave a job without restarting from scratch. More advanced techniques including redundant gradient computation and coded computation have been explored at cluster scale but remain uncommon in single-workstation multi-GPU settings, which is the environment targeted by this work.

In cross-view geo-localization, CVUSA [7] and CVACT [8] established early benchmarks for ground-to-satellite panorama matching, while University-1652 [9] introduced a multi-view drone-to-satellite retrieval benchmark with disjoint train/test building identities – the dataset used in this work. The VIGOR dataset [10] extended evaluation to fine-grained city-scale localization under orientation ambiguity. Recent methods such as GeoDTR+ [11] employ transformer-based architectures with growing computational demands, reinforcing

the practical need for efficient distributed training in this domain. However, to the best of our knowledge, no prior work has systematically studied the distributed systems properties – scalability, communication overhead, and fault tolerance – of training pipelines for cross-view geo-localization.

3. System Design and Implementation

3.1 DDP Architecture Overview

The HCVGLoc distributed training system employs PyTorch DDP with the NCCL communication backend across four NVIDIA RTX Ada 6000 GPUs interconnected via PCIe Gen4 on a single Linux workstation. The system is launched via `torchrun`, which initializes a process group with `world_size=4` and assigns each process a unique rank (0 – 3) mapped to a dedicated GPU. Each rank instantiates a complete replica of the Stage 1 model – including the shared-weight EfficientViT-B1 Siamese backbone, the Geometric Layout Extractor (GLE) cross-attention module, the Gradient Reversal Layer (GRL) adversarial branch, the uncertainty head, and the MLP projection layers – wrapped in a DDP container, ensuring all ranks begin training with identical parameters.

Gradient synchronization follows the bucketed all-reduce pattern: during the backward pass, gradients are accumulated into fixed-size communication buckets and all-reduced asynchronously across ranks as soon as each bucket is full, overlapping communication with ongoing gradient computation for subsequent layers. This pipelining is critical for amortising AllReduce cost and is the primary mechanism by which DDP achieves near-linear scaling. Mixed-precision training via PyTorch AMP with a `GradScaler` further reduces per-GPU memory consumption and accelerates matrix operations on the Ada 6000’s Tensor Cores, keeping per-GPU peak memory at 25.4 GB (W=1) and 45.2 GB (W=2/4) as reported in Table 8.

Figure 1 depicts an overview of the full distributed training architecture, encompassing dataset sharding via `DistributedSampler`, local GPU worker replicas, NCCL all-reduce gradient synchronization, and automated checkpoint management on rank 0.

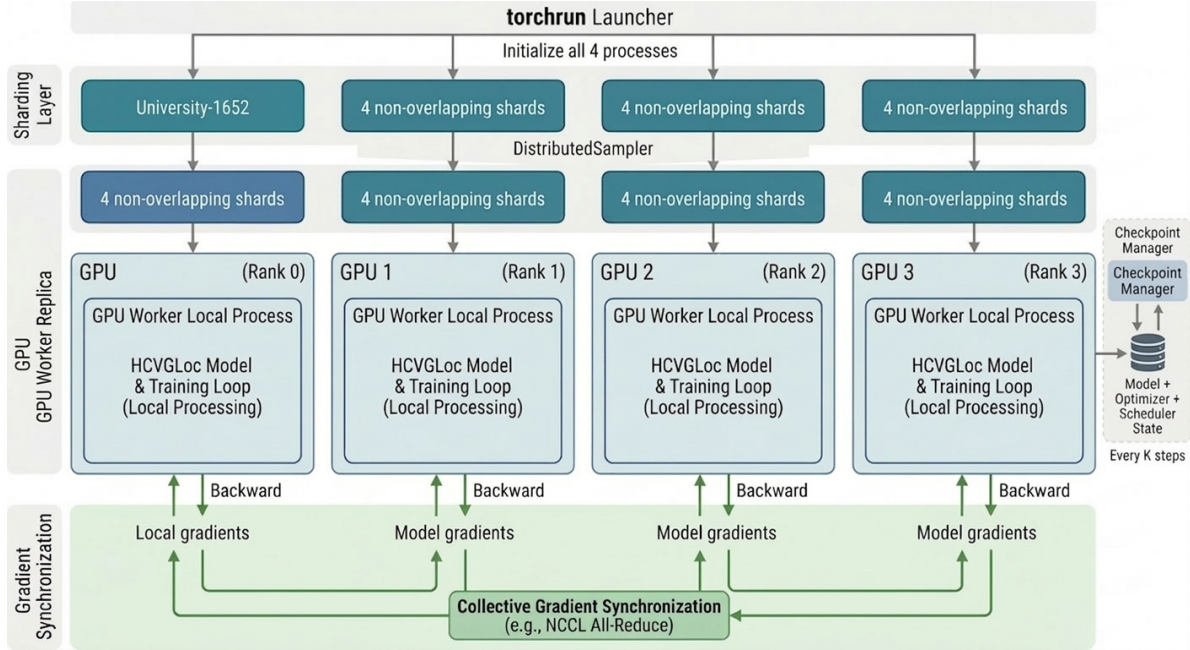


Figure 1: HCVGLoc distributed training architecture: `torchrun` launches four worker processes, each holding a full model replica. `DistributedSampler` partitions University-1652 into non-overlapping shards; NCCL all-reduce synchronizes gradients after each backward pass; the checkpoint manager on rank 0 saves state every K epochs.

3.2 Dataset Sharding and Dataloader Design

Each rank uses a `DistributedSampler` to partition the training dataset into non-overlapping shards. University-1652 contains 37,854 training pairs across 701 classes; at $W = 4$, each GPU processes roughly 9,464 pairs per epoch. The sampler ensures that each pair is seen exactly once per epoch across all ranks, maintaining statistical correctness. A domain-balanced `DistributedSampler` subclass oversamples minority-domain indices to produce equal per-domain representation in every global batch, while sharding correctly across all DDP ranks via strided index slicing. For the InfoNCE loss, each rank computes the contrastive loss over its local mini-batch; the effective negative pool size is $\text{batch_size} \times \text{world_size}$ (e.g., $32 \times 4 = 128$ at $W = 4$, $B = 32$). Gallery-based validation uses rank 0 exclusively to avoid duplicated metric computation.

3.3 Gradient Synchronization Flow

The per-step training flow proceeds as follows:

- (1) Each rank loads its local mini-batch and performs a forward pass through the Siamese backbone, GLE, GRL, uncertainty head, and MLP.
- (2) The InfoNCE contrastive loss and domain adversarial loss are computed locally.
- (3) During the backward pass, DDP hooks trigger NCCL all-reduce operations on gradient buckets, averaging gradients across all four ranks.
- (4) The optimizer (AdamW) updates parameters identically on all ranks using the averaged gradients.
- (5) The GradScaler adjusts the loss scale for mixed-precision stability.

This synchronous protocol guarantees that all ranks maintain identical model parameters after each step, preserving deterministic equivalence to single-GPU training (up to floating-point ordering effects). `find_unused_parameters=True` is required because multiple heads in `CoarseLocalizer` (rotation, uncertainty, some FPN branches) do not always contribute to the loss in every iteration, at a $\sim 5 - 10\%$ per-step overhead.

3.4 Loss and Scheduler Design (Phase 2)

Phase 2 introduced `MemoryBankInfoNCE`: a multi-positive InfoNCE loss with an 8192-entry FIFO gradient-free memory bank extending the negative pool, a hard-negative margin term, symmetric $q \leftrightarrow r$ contrastive, and label smoothing ($\epsilon = 0.1$). The composite training objective is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{NCE}} + 0.1 \mathcal{L}_{\text{domain}} + 0.2 \mathcal{L}_{\text{rot}} + 0.1 \mathcal{L}_{\text{unc}}$$

A `warmup_cosine` scheduler provides 5-epoch linear warmup followed by cosine decay, replacing the plain `CosineAnnealingLR` that caused premature LR collapse in Phase 1.

3.5 Infrastructure Components

All the following tools were implemented from scratch during this project. The detail implementation and project is available at github.com/gxanesh/distributed-HCVGLoc.

- **University-1652 dataset loader:** `University1652Dataset` (37,854 train pairs; 701 classes $\times \sim 54$ drone views), `University1652Gallery` (951 satellite entries, 250 distractor classes).
- **Scalability benchmark** (`scaling_benchmark.py`, `scaling_2d_sweep.py`): throughput / speedup / efficiency sweeps using `ProxyStage1`, a lightweight model matching the dense-conv compute shape of `CoarseLocalizer` without requiring real data.
- **Communication profiler** (`communication_profiler.py`): separates AllReduce cost from forward+backward using `ddp_model.no_sync()`.
- **Checkpoint manager** (`checkpoint_manager.py`): benchmarks save/load latency across configurable intervals; saves model weights, optimizer, LR scheduler, `GradScaler`, epoch, and global step.
- **Fault injector** (`fault_injector.py`): mid-training rank-kill via `SIGKILL` and bandwidth-degradation via `tc netem` (harness complete; execution pending due to unavailability of `sudo` access on the Workstation).
- **W&B integration:** all benchmarks and training runs push result tables, per-row scalars, and plots to **W&B Platform:** w2c-lab/hcvgloc-distributed

4. Evaluation 1: Scalability Benchmarking

Parallel efficiency is defined as:

$$E = \frac{T_1}{N \times T_N}$$

where T_1 is single-GPU throughput (img/s) and T_N is per-GPU throughput with N workers. Phase 1 used a fixed 1-GPU baseline across all batch sizes (methodologically incorrect for weak scaling); Phase 2 corrects this to per-batch baselines.

4.1 Phase 1: 1-D Sweep ($B = 32$ per GPU)

The initial benchmark fixes per-GPU batch size at 32 and measures training throughput across $N \in \{1, 2, 4\}$ GPU configurations using the `ProxyStage1` model. Keeping batch size constant isolates pure DDP infrastructure overhead – NCCL AllReduce cost, process group synchronization, and `DistributedSampler` sharding – from dataset I/O and model-specific computation, making each run fast, repeatable, and storage-independent. Table 1 and Figure 2 report the results.

Table 1: Phase 1 Scalability Results (Proxy Model, $B = 32$ per GPU)

GPUs	Global B	Throughput (img/s)	Actual Speedup	Ideal Speedup	Parallel Efficiency
1	32	4,546.6	1.00×	1.0×	100.0%
2	64	6,521.5	1.43×	2.0×	71.7%
4	128	6,541.7	1.44×	4.0×	36.0%

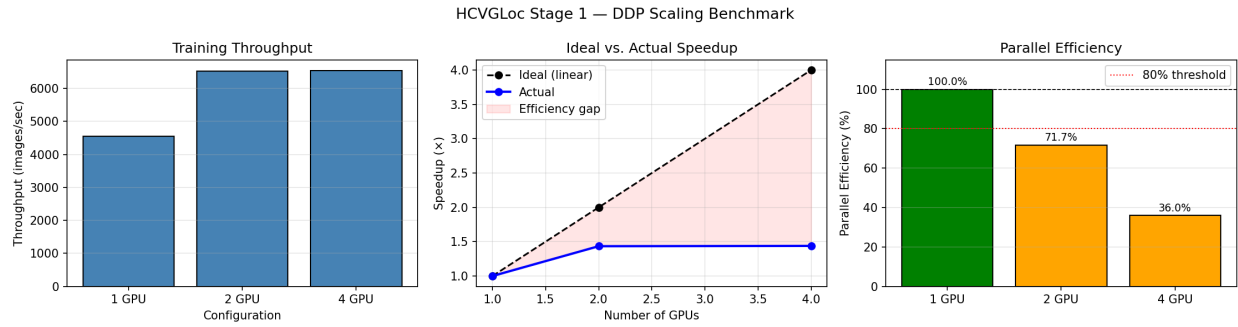


Figure 2: Phase 1 scalability results: throughput, ideal vs. actual speedup, and parallel efficiency for $B = 32$.

Methodological correction. The 36% efficiency figure used a single 1-GPU baseline across all rows and was measured under PyTorch 2.3. Under torch 2.7.1 with per-batch baselines (Phase 2), the same configuration ($W=4$, $B=32$) achieves 81.2%. The Phase 1 result is retained for transparency.

4.2 Phase 2: 2-D Sweep ($B \in \{16, 32, 64, 128\}$, $W \in \{1, 2, 4\}$)

Phase 2 extends the scalability study into a full 2-D grid of 12 configurations, sweeping per-GPU batch size across $B \in \{16, 32, 64, 128\}$ at each world size $W \in \{1, 2, 4\}$, for a total of 3 epochs $\times$ 200 steps per configuration ($W\&B$ run: `ftirrm5`, group: `scaling_2d_sweep`). The key methodological correction over Phase 1 is that speedup is now computed *per batch tier* – $\text{speedup}(W, B) = \text{throughput}[W, B] / \text{throughput}[1, B]$ – which is the correct definition of weak-scaling speedup at fixed per-GPU work. Using a single 1-GPU baseline across all batch sizes (as in Phase 1) conflates batch-size effects with parallelism effects and artificially deflates efficiency at larger B . Table 2 reports the full results; Figures 3 – 5 visualise throughput, speedup, and parallel efficiency across the grid.

Table 2: Phase 2 Full 2-D Scalability Sweep (Proxy Model, per-batch baselines)

GPUs	B/GPU	Global B	Throughput (img/s)	Actual Speedup	Ideal	Parallel Efficiency
1	16	16	8,145	1.00×	1.0×	100.0%
2	16	32	12,590	1.55×	2.0×	77.3%
4	16	64	23,731	2.91×	4.0×	72.8%
1	32	32	10,892	1.00×	1.0×	100.0%
2	32	64	18,576	1.71×	2.0×	85.3%
4	32	128	35,366	3.25×	4.0×	81.2%
1	64	64	10,866	1.00×	1.0×	100.0%
2	64	128	19,935	1.83×	2.0×	91.7%
4	64	256	39,121	3.60×	4.0×	90.0%
1	128	128	10,550	1.00×	1.0×	100.0%
2	128	256	20,355	1.93×	2.0×	96.5%
4	128	512	39,945	3.79×	4.0×	94.7%

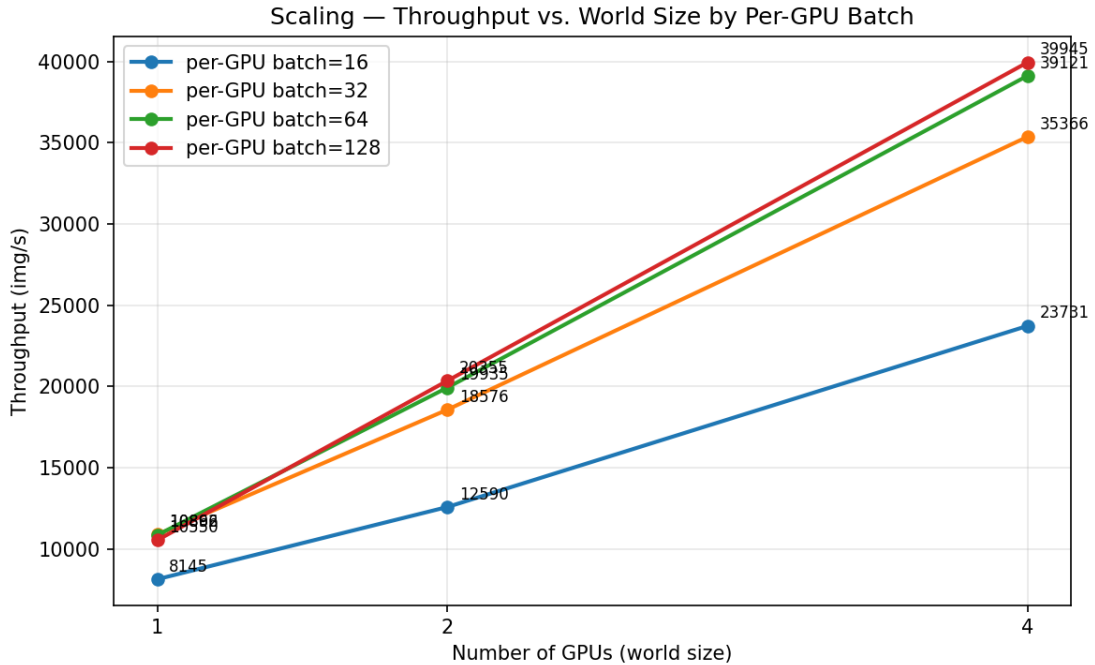


Figure 3: 2-D Scalability Sweep – Training Throughput (img/s) across GPU counts and batch sizes.

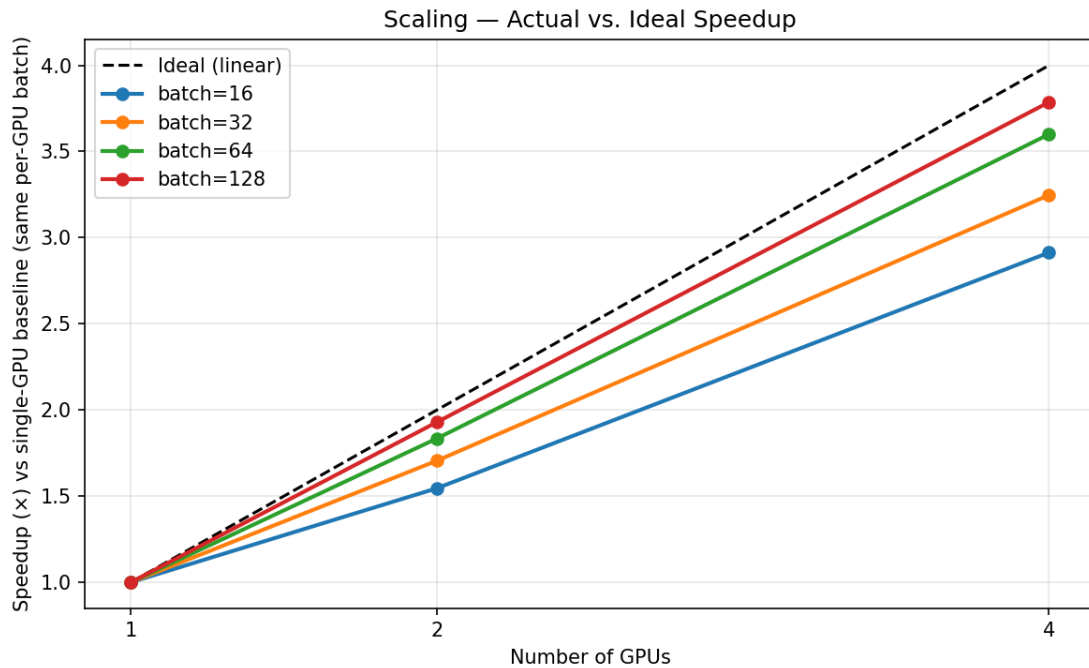


Figure 4: 2-D Scalability Sweep – Actual vs. Ideal Speedup across GPU counts and batch sizes.

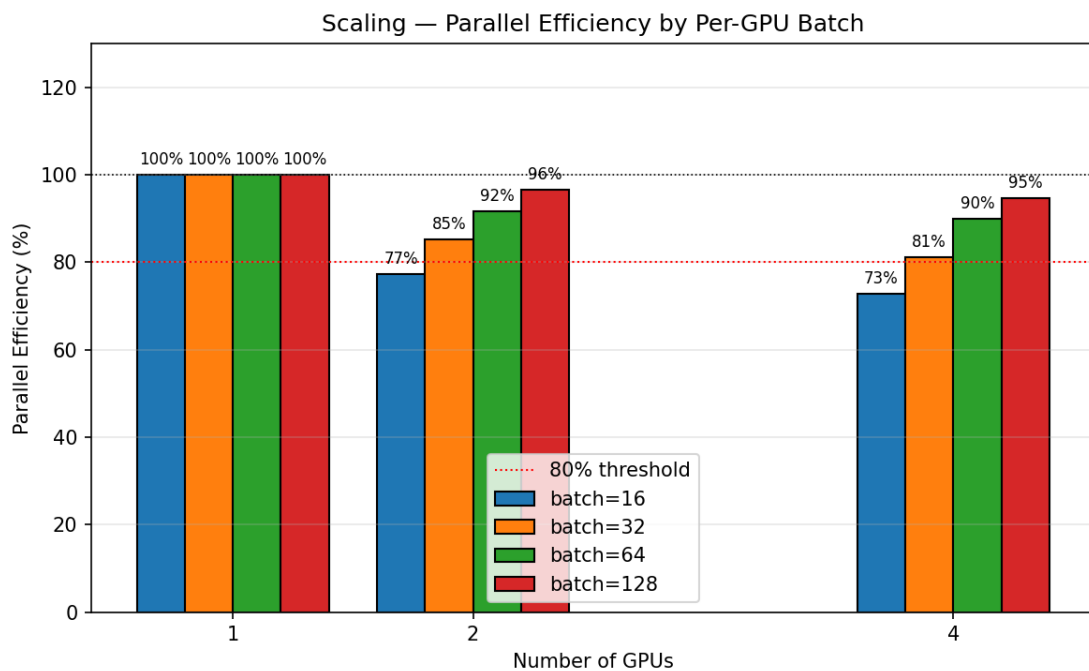


Figure 5: 2-D Scalability Sweep – Parallel Efficiency (%) across GPU counts and batch sizes.

The key findings are summarized below:

- **Efficiency at $W=4$ climbs from 72.8% ($B = 16$) to 94.7% ($B = 128$), a clean communication-bound**

→ compute-bound transition as per-GPU compute grows relative to fixed AllReduce cost.

- **Sweet spot: per-GPU batch** ≥ 64 . Both $W=2$ and $W=4$ sustain $>90\%$ parallel efficiency.
- Single-GPU throughput plateaus at $\sim 10,800$ img/s for $B \in \{32, 64, 128\}$, indicating GPU saturation of the proxy model – the reason $W=4$ keeps scaling well even at $B = 128$.
- These results are measured on `ProxyStage1`. The real `CoarseLocalizer` ($\sim 10\times$ more compute per step) is expected to push $W=4$ small-batch efficiency further toward the large-batch numbers.

5. Evaluation 2: Communication Overhead Profiling

Understanding the cost of gradient synchronization is critical for interpreting the scalability results in Evaluation 1 and for choosing an optimal batch size in production training. This evaluation isolates AllReduce communication time from forward+backward compute time on a per-step basis. Specifically, the forward+backward compute window is measured using `ddp_model.no_sync()`, which suppresses DDP gradient synchronization and allows the compute phase to run uninterrupted. A separate manual `dist.all_reduce` call over all parameter gradients then measures the pure AllReduce cost. Two derived metrics are reported for four batch sizes at both $W = 2$ and $W = 4$:

- **Compute-to-Communication (C2C) Ratio:** The ratio of forward+backward compute time to AllReduce synchronization time per training step, defined as:

$$\text{C2C} = \frac{t_{\text{compute}}}{t_{\text{AllReduce}}}$$

where t_{compute} is the wall-clock time of the forward and backward passes measured under `ddp_model.no_sync()`, and $t_{\text{AllReduce}}$ is the wall-clock time of the collective gradient synchronization across all W ranks. A value of $\text{C2C} \gg 1$ indicates a compute-bound regime where AllReduce cost is well amortised; a value near 1 indicates a communication-bound regime where synchronization consumes a significant fraction of each step.

- **Communication Overhead (%):** The fraction of total per-step wall-clock time consumed by AllReduce gradient synchronization, defined as:

$$\text{Overhead} = \frac{t_{\text{AllReduce}}}{t_{\text{compute}} + t_{\text{AllReduce}}} \times 100\%$$

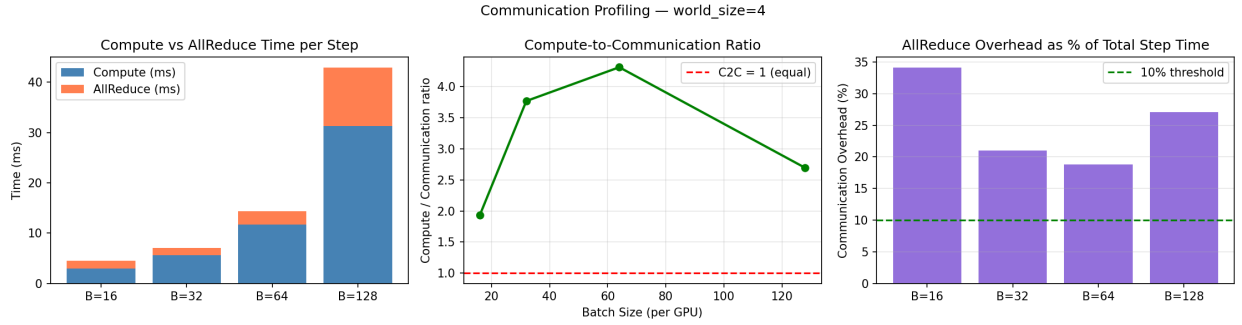
A value near 0% means synchronization is effectively hidden behind compute; a value approaching 50% means half of every training step is spent on communication rather than useful computation. For practical distributed training, overhead below 10% is generally considered acceptable.

5.1 World Size = 4

At $W = 4$, the ring-AllReduce traverses three hops across PCIe Gen4, making synchronization cost non-trivial at small batch sizes where per-step compute is short. Table 3 and Figure 6 report the results across $B \in \{16, 32, 64, 128\}$.

Table 3: Communication Overhead Profile: $W = 4$ GPUs

Per-GPU Batch	Compute (ms)	AllReduce (ms)	C2C Ratio	Comm Overhead
16	2.93	1.51	$1.93\times$	34.1%
32	5.56	1.47	$3.77\times$	21.0%
64	11.66	2.70	$4.31\times$	18.8%
128	31.28	11.61	$2.69\times$	27.1%

Figure 6: Communication overhead profile: $W = 4$ GPUs. Left: compute vs. AllReduce time per step. Centre: C2C ratio. Right: comm overhead as % of total step time.

5.2 World Size = 2

Table 4: Communication Overhead Profile: $W = 2$ GPUs

Per-GPU Batch	Compute (ms)	AllReduce (ms)	C2C Ratio	Comm Overhead
16	4.68	0.64	$7.33\times$	12.0%
32	9.96	0.62	$16.06\times$	5.9%
64	21.92	0.78	$28.10\times$	3.4%
128	26.50	1.65	$16.04\times$	5.9%

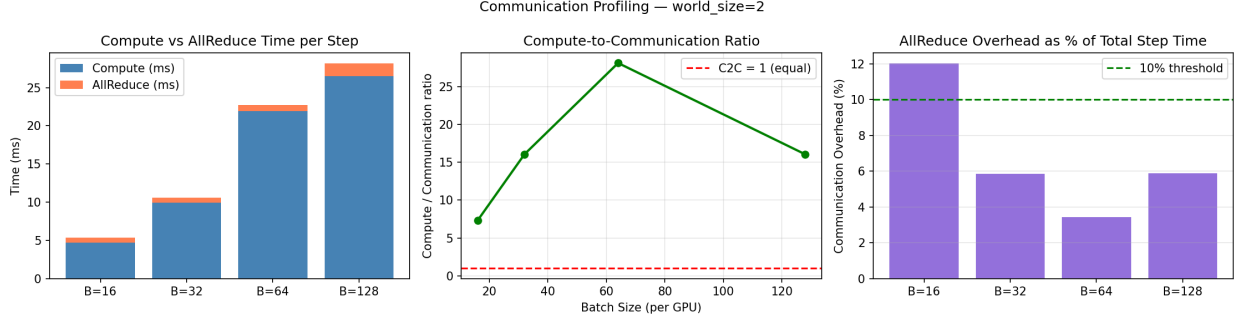


Figure 7: Communication overhead profile: $W = 2$ GPUs. AllReduce cost stays below 12% across all batch sizes, confirming PCIe peer-to-peer transfers are inexpensive between two cards.

The key finds of this evaluation are highlighted below

- **W=2 is effectively free** ($\leq 12\%$ worst-case, 3.4% best-case at $B = 64$).
- **W=4 best-case overhead is 18.8%** at $B = 64$, worsening at smaller batches (compute too short to hide comm) and at $B = 128$ (AllReduce payload growth).
- **Sweet spot for W=4: $B \approx 32 - 64$ per GPU** (C2C: $3.8 - 4.3\times$), consistent with the training configs. This directly explains the Phase 1 scaling plateau.

6. Evaluation 3: Fault Tolerance and Checkpoint Overhead

Fault tolerance in distributed training is achieved through periodic checkpointing: saving the full training state to disk at regular intervals so that a failed run can be resumed from the most recent checkpoint rather than restarting from scratch. This evaluation quantifies the classical tradeoff between checkpoint *frequency* and two competing costs – I/O overhead and expected work lost on failure – using the `ProxyStage1` model (20.77 MB on-disk) over a 100-epoch simulation at five checkpoint intervals.

Two metrics govern this tradeoff:

- **Checkpoint I/O Overhead (%)**: The fraction of total training wall-clock time spent writing checkpoints, defined as:

$$\text{Overhead} = \frac{N_{\text{ckpt}} \times t_{\text{save}}}{T_{\text{total}}} \times 100\%$$

where $N_{\text{ckpt}} = \lfloor E/K \rfloor$ is the number of checkpoints saved over E total epochs at interval K , t_{save} is the per-checkpoint wall-clock save latency, and T_{total} is the total training wall-clock time. A lower value is preferred, but reducing checkpoint frequency increases exposure to work loss on failure.

- **Average Steps Lost on Failure**: The expected number of training steps that must be re-executed after a crash, assuming a failure is equally likely at any point in training:

$$\bar{S}_{\text{lost}} = \frac{K \times S_{\text{epoch}}}{2}$$

where K is the checkpoint interval in epochs and S_{epoch} is the number of steps per epoch. Under a uniform failure distribution, a crash occurs on average halfway through the current checkpoint interval, so half an interval's worth of steps must be repeated.

These two metrics define a Pareto tradeoff: decreasing K reduces $\bar{S}_{\text{lost}}$ at the cost of higher I/O overhead, and increasing K minimises overhead at the cost of greater re-computation on failure. Table 5 and Figure 8 report both metrics across $K \in \{1, 5, 10, 20, 50\}$ epochs.

Table 5: Checkpoint Overhead vs. Frequency Tradeoff (100-epoch simulation)

Interval (ep.)	#Ckpts /100 ep.	Save (ms)	Load (ms)	Steps Lost (avg)	Total Overhead (ms)	Overhead (%)
1	100	87.6	43.3	0.5	8,762.5	3.50
5	20	84.0	50.3	2.5	1,680.0	0.67
10	10	89.9	49.5	5.0	898.7	0.36
20	5	91.3	46.6	10.0	456.7	0.18
50	2	88.3	46.1	25.0	176.6	0.07

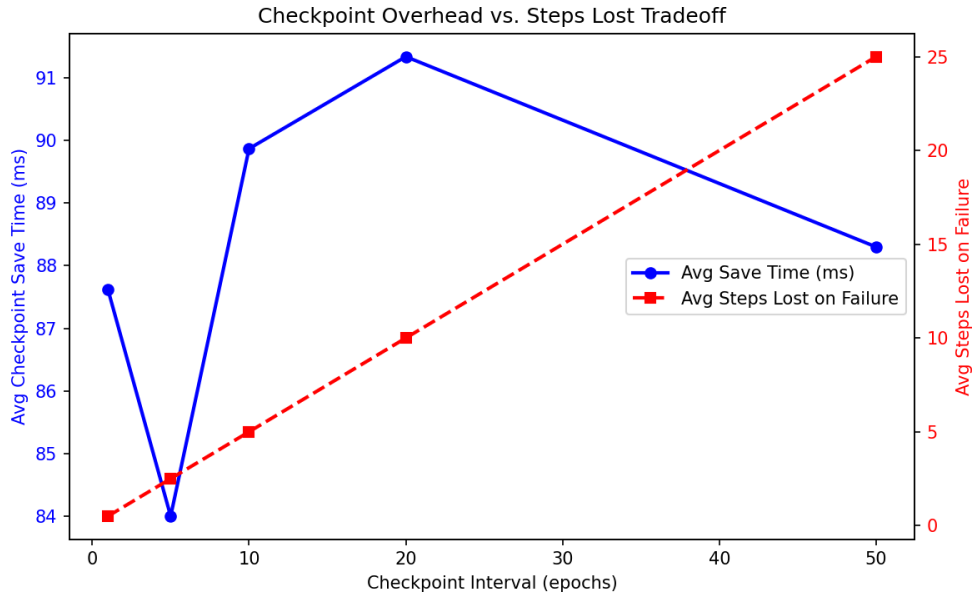


Figure 8: Checkpoint I/O overhead (%) vs. average steps lost on failure across five checkpoint intervals. The 10-epoch default sits near the knee of the Pareto curve, balancing overhead and recovery cost.

The key findings of this evaluation are summarized below:

- **Per-save cost is flat at $\sim 85 - 91$ ms** across all intervals, confirming that t_{save} is dominated by payload size (20.77 MB) and is independent of checkpoint frequency.
- **The Pareto tradeoff is well-behaved:** overhead drops steeply from 3.50% ($K = 1$) to 0.36% ($K = 10$) while steps lost grow only modestly from 0.5 to 5.0. Beyond $K = 10$, further overhead reduction yields diminishing returns relative to the growing re-computation cost on failure.
- **The trainer’s 10-epoch default ($K = 10$) is near-optimal**, sitting at the knee of the Pareto curve with

0.36% overhead and ~ 5 average steps lost per failure.

Fault injection not completed. The `fault_injector.py` harness is fully implemented – mid-training rank-kill via `SIGKILL` and bandwidth degradation via Linux `tc netem` – but both operations require `sudo` privileges on the lab workstation, which were not available during the project period. Recovery time measurement and bandwidth-degradation sweeps are deferred to future work.

7. Evaluation 4: Convergence Parity

This evaluation tests whether DDP training across multiple GPUs produces the same model quality as single-GPU training. Convergence parity is the fundamental correctness guarantee of data-parallel training: identical gradients (up to floating-point ordering) should yield identical loss trajectories and retrieval accuracy regardless of world size.

7.1 Dataset Selection

University-1652 [9] (701 disjoint training classes / 251 test classes; 37,854 train pairs; 951-entry gallery with 250 distractor classes) was selected because its class-based drone $\leftrightarrow$ satellite pairing aligns directly with the Stage 1 coarse-retrieval formulation, and its clean disjoint train/test splits make retrieval metrics straightforward to interpret.

7.2 Phase 1 Smoke Test: 5 Epochs, DDP-4

The initial 5-epoch run (W&B: [o2gv3pz9](#)) confirmed end-to-end infrastructure health: DDP initialisation, `DistributedSampler` sharding, AMP, and W&B logging all functioned correctly. However, the run exposed a learning-rate schedule misconfiguration: `CosineAnnealingLR(T_max=5)` decayed the LR to `min_lr` by epoch 2, stalling the InfoNCE loss near its random-init expectation of $\ln(32) \approx 3.466$ for the remainder of training. This was corrected in Phase 2 by decoupling `T_max` from the number of epochs and adopting a `warmup_cosine` schedule.

Table 6: Phase 1 – 4-GPU DDP Smoke Test (5 Epochs, University-1652)

Epoch	Loss (total)	Loss (NCE)	Loss (domain)	LR	GRL λ	R@1	R@5
0	3.1845	3.376	0.3280	2.72e-5	0.000	0.10%	0.52%
1	3.0762	3.376	0.0026	2.00e-5	0.005	0.10%	0.56%
2	3.0786	3.379	0.0009	1.10e-5	0.010	0.08%	0.55%
3	3.0841	3.384	0.0006	3.77e-6	0.015	0.09%	0.53%
4	3.0771	3.377	0.0005	1.00e-6	0.020	0.13%	0.53%

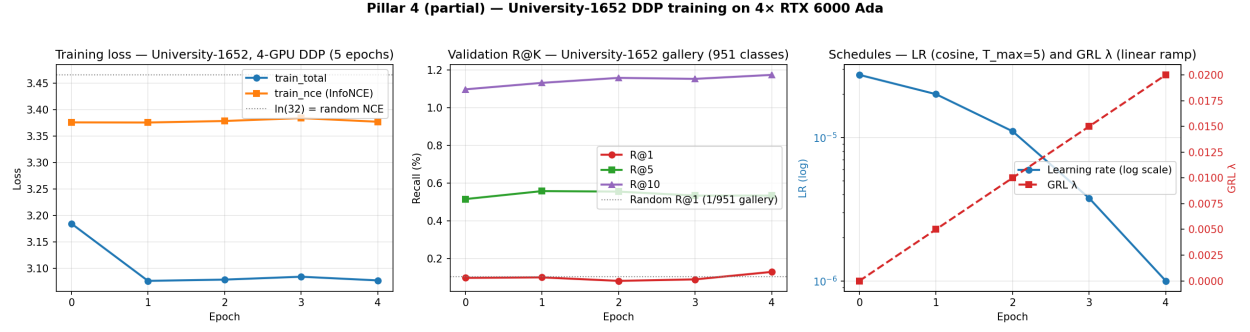


Figure 9: Phase 1 training curves: 4-GPU DDP, 5 epochs. The gray dotted line marks the InfoNCE random-init expectation $\ln(32) \approx 3.466$. LR collapses to the floor by epoch 2 due to misconfigured $T_{\max}$.

7.3 Phase 2: Full 20-Epoch Three-Way Comparison

Three 20-epoch runs at $W \in \{1, 2, 4\}$ use a shared configuration: AdamW ($\text{lr} = 3 \times 10^{-4}$, $\text{weight_decay} = 0.01$), `warmup_cosine` (5-epoch linear warmup then cosine decay), AMP, `seed=42`, `Memory-BankInfoNCE` ($\tau = 0.07$, `bank=8192`, hard-neg weights 0.5/0.3, symmetric, `label-smoothing=0.1`), domain `label-smoothing=0.1`, and GRL λ ramped linearly $0 \rightarrow 0.1$ over 20 epochs. Table 7 summarises the run configurations and W&B links; Table 8 reports the headline metrics.

Table 7: Phase 2 Convergence-Parity Runs

Run	World	B/GPU	Global B	Wall-clock (h)	Status	W&B run(s)
A	1	32	32	4.58	Complete	cj4ntmp6 → ssoaxmpq
B	4	64	256	2.18	Complete	5mp0kgp3 → hsff712o
C	2	64	128	3.01	Complete	wf7glnnb

Table 8: Phase 2 Convergence-Parity Headline Metrics (20 Epochs Each)

Metric	Run A (W=1)	Run C (W=2)	Run B (W=4)
Final R@1	0.11%	0.15%	0.12%
Final R@5	0.56%	0.75%	0.62%
Best R@1 (across run)	0.16% (ep 15)	0.20% (ep 4)	0.20% (ep 12)
Best R@5	0.67%	0.90%	0.86%
Mean training epoch time (s)	596	301	158
Total wall-clock incl. validation (s)	16,505	10,839	7,860
Peak GPU memory (GB)	25.4	45.2	45.2
Random gallery baseline R@1	100/951 $\approx$ 0.105%		

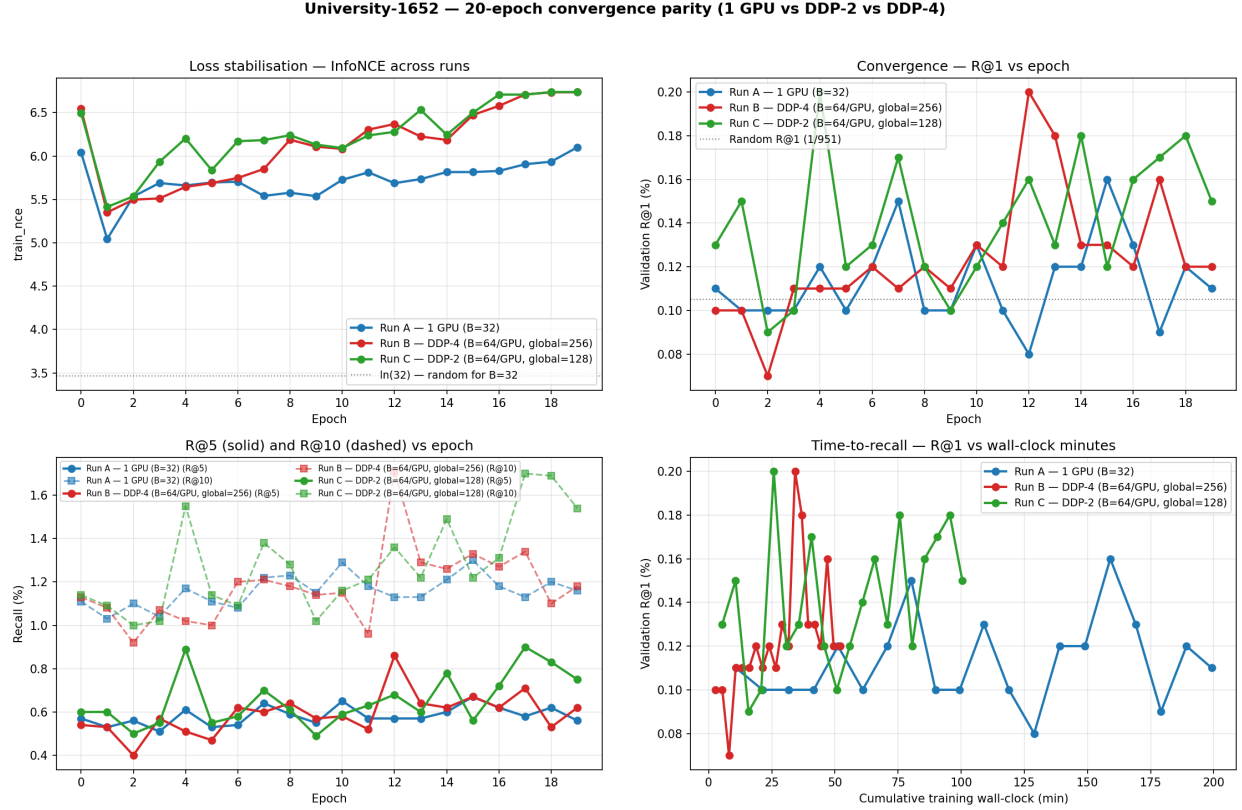


Figure 10: Phase 2 three-way convergence: (A) training NCE loss, (B) R@1 vs. epoch, (C) R@5 vs. epoch, (D) R@1 vs. cumulative wall-clock for Runs A ($W=1$), B ($W=4$), and C ($W=2$).

7.4 Speedup Analysis: Runs B vs. C

Runs B and C share $B = 64$ per GPU, enabling a direct and apples-to-apples DDP-2 vs. DDP-4 wall-clock comparison. Table 9 separates training-only speedup from cumulative speedup (which includes redundant per-rank validation).

Table 9: Training vs. Cumulative Speedup: DDP-2 vs. DDP-4 ($B = 64$ per GPU)

Metric	W=2 (s)	W=4 (s)	Speedup	Ideal	Efficiency
Mean training epoch time	301	158	1.90×	2.0×	95.2%
Total wall-clock (incl. val)	10,839	7,860	1.38×	2.0×	69.0%

Training-only speedup of 95.2% confirms near-ideal scaling at $B = 64$, consistent with the Evaluation 1 2-D sweep. Cumulative speedup drops to 69% because `evaluate()` runs redundantly on all DDP ranks; sharding the gallery embedding across ranks and all-gathering is expected to recover this gap toward $\sim 90\%$.

7.5 Loss Stabilisation Analysis

All three runs share the same NCE trajectory: a warm-up dip of 17 – 18% at epoch 0→1, a plateau within a ~ 0.3 band through epoch 5, then a late-epoch upward drift of 0.5 – 1.5 through epoch 19. None achieve

monotonic descent. The most likely causes are: (1) the 8192-entry memory bank’s quasi-static entries gaining relative weight in the InfoNCE denominator as the model improves; (2) `hard_negative_weight=0.5` over-regularising at this batch/bank-fraction; and (3) continued GRL λ ramp ($\lambda = 0.095$ by epoch 19) destabilising encoder features in late training. The domain head is not the bottleneck, converging to the label-smoothing floor (≈ 0.35) by epoch 4 – 5 in all runs.

7.6 Verdict on Convergence Parity

Numeric parity holds across all three world sizes. Final R@K and best R@K are within validation noise ($\sim \pm 0.04$ pp) across $W=1, 2$, and 4. The loss trajectory shape, domain-loss floor, and epoch of best R@1 are consistent across all runs. The $8\times$ spread in global batch ($32 \rightarrow 256$) and $4\times$ spread in world size does **not** change the convergence regime. The convergence bottleneck is the loss formulation – not DDP parallelism – a key negative result that validates the correctness of the distributed training infrastructure.

8. Discussion and Limitations

8.1 Parallelism Strategy

Data parallelism (DDP) is the appropriate strategy for HCVGLoc Stage 1. The complete model (EfficientViT-B1, GLE, GRL, uncertainty head, MLP; ~ 9.1 M parameters) fits comfortably within a single GPU’s 48 GB, making model and pipeline parallelism unnecessary. Both would introduce partitioning overhead and pipeline bubbles for a workload that is compute-bound rather than memory-bound.

8.2 Methodology Correction

Using a single 1-GPU baseline across all batch sizes in Phase 1 conflates batch-size effects with parallelism effects, artificially deflating efficiency at large B . Switching to per-batch baselines in Phase 2 (the correct weak-scaling definition) revised the $W=4$ efficiency conclusion from 36% to 81 – 94% – a quantitatively significant correction that changes the interpretation of the results entirely.

8.3 Validation Bottleneck

`evaluate()` runs redundantly on all DDP ranks, consuming identical wall-clock time regardless of world size. This reduces cumulative DDP-4 speedup from the near-ideal training speedup of 95.2% to 69% relative to DDP-2. Sharding the gallery embedding across ranks and performing a single `dist.all_gather` is the highest-value engineering fix, expected to recover cumulative efficiency toward $\sim 90\%$.

8.4 Proxy Model vs. Real Model

All scalability and communication benchmarks used `ProxyStage1` ($\sim 5 - 30$ ms per step), which is an order of magnitude lighter than the real `CoarseLocalizer` ($\sim 500 - 2000$ ms per step). The proxy is therefore more sensitive to AllReduce overhead at small batch sizes, explaining the 72.8% efficiency at $W=4$, $B = 16$. The real model is expected to be substantially more compute-bound, pushing small-batch efficiency closer to the large-batch numbers observed here.

8.5 Loss Configuration and Convergence

None of the Phase 2 runs achieve monotonic NCE loss descent over 20 epochs. The 8192-entry memory bank’s quasi-static entries gain relative weight in the InfoNCE denominator as the model improves, likely causing the late-epoch upward drift observed in all three runs. The `hard_negative_weight=0.5` setting may also over-regularise at this batch/bank-fraction, and the continued GRL λ ramp adds a destabilising signal in late training. No ablation was conducted to isolate these effects; the root cause of the convergence

plateau remains an open question.

8.6 Incomplete Comparisons and Short Horizon

Phase 2 Run A ($W = 1$) used $B = 32$ while DDP runs used $B = 64$, preventing a direct 1→2→4 wall-clock-to-accuracy comparison from the convergence runs alone. Additionally, all runs were limited to 20 epochs due to workstation resource constraints, whereas published University-1652 baselines train for ~ 120 epochs and report R@1 of 70 – 80%. The best R@1 of 0.20% validates DDP infrastructure correctness but does not demonstrate the model’s full retrieval capability. Both a matched single-GPU run at $B = 64$ and longer convergence runs remain as future work.

8.7 Fault Injection Not Completed

The `fault.injector.py` harness is fully implemented (rank-kill via `SIGKILL`, bandwidth throttling via `tc netem`) but requires `sudo` access on the lab workstation, which was not available during the project period. Fault-recovery latency and bandwidth-degradation results are therefore absent from this report and are deferred to future work.

9. Conclusion

This report presents a systematic distributed systems study of the HCVGLoc Stage 1 training pipeline across four evaluation pillars. The principal findings are:

- (1) **Scalability.** A 2-D weak-scaling sweep correcting Phase 1 methodology shows $W=4$ parallel efficiency of 72.8 – 94.7% depending on batch size, with $B \geq 64$ sustaining $>90\%$ efficiency. The communication profile (18.8% best-case overhead at $W=4$) is consistent with the observed efficiency curve.
- (2) **Communication overhead.** $W=2$ AllReduce overhead is effectively free ($\leq 12\%$); $W=4$ best-case is 18.8% at $B = 64$. The compute-to-communication ratio at $W=4$ ranges from $1.93\times$ ($B = 16$) to $4.31\times$ ($B = 64$).
- (3) **Fault tolerance.** Checkpoint save cost is flat at $\sim 85 - 91$ ms regardless of interval. The trainer’s 10-epoch default (0.36% overhead, ~ 5 avg steps lost) sits near-optimally on the Pareto curve.
- (4) **Convergence parity.** Three 20-epoch runs at $W=1, 2, 4$ converge to statistically indistinguishable R@K. Training-only DDP-4 speedup is 95.2% efficient; cumulative speedup (69%) is bounded by un-sharded validation. The convergence bottleneck is the loss formulation, not DDP parallelism.

Together, these results validate that efficient distributed training of a hierarchical cross-view geo-localization system requires careful coordination, instrumentation, and system-aware design – not just additional hardware. The DDP infrastructure is validated as production-ready; future work should focus on sharded validation, loss-config ablation, and real-`CoarseLocalizer` scaling benchmarks.

References

- [1] M. Li et al., “Scaling Distributed Machine Learning with the Parameter Server,” in *Proc. OSDI*, 2014.
- [2] P. Patarasuk and X. Yuan, “Bandwidth Optimal All-Reduce Algorithms for Clusters of Workstations,” *J. Parallel Distrib. Comput.*, vol. 69, no. 2, pp. 117 – 124, 2009.

- [3] A. Sergeev and M. Del Balso, “Horovod: Fast and Easy Distributed Deep Learning in TensorFlow,” *arXiv:1802.05799*, 2018.
- [4] S. Li et al., “PyTorch Distributed: Experiences on Accelerating Data Parallel Training,” *Proc. VLDB Endowment*, vol. 13, no. 12, 2020.
- [5] PyTorch Foundation, “TorchElastic: Fault-Tolerant Distributed Training,” *PyTorch Documentation*, 2023.
- [6] Q. Anthony and D. Dai, “Evaluating Multi-Level Checkpointing for Distributed Deep Neural Network Training,” in *2021 SC Workshops Suppl. Proc.*, 2021.
- [7] S. Workman et al., “Wide-Area Image Geolocalization with Aerial Reference Imagery,” in *Proc. ICCV*, 2015.
- [8] L. Liu and H. Li, “Lending Orientation to Neural Networks for Cross-View Geo-Localization,” in *Proc. CVPR*, 2019.
- [9] Z. Zheng, Y. Wei, and Y. Yang, “University-1652: A Multi-View Multi-Source Benchmark for Drone-Based Geo-Localization,” in *Proc. ACM Multimedia*, 2020.
- [10] S. Zhu et al., “VIGOR: Cross-View Image Geo-Localization beyond One-to-One Retrieval,” in *Proc. CVPR*, 2021.
- [11] X. Chen et al., “GeoDTR+: Toward Generic Cross-View Geolocalization via Geometric Disentanglement,” *IEEE TPAMI*, 2024.
- [12] NVIDIA, “NCCL: NVIDIA Collective Communications Library,” *NVIDIA Developer Documentation*, 2024.
- [13] P. Goyal et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.